VELLORE INSTITUTE OF TECHNOLOGY

Vellore, Tamil Nadu, India

FINAL PROJECT REPORT

EXPLICITE AI

An AI-Powered Explainable Code Review and Security Analysis Platform

---

Submitted by:

Dhruv Sharma (24BBS0063)

Rijul Kothawade (24BBS0230)

Submitted in partial fulfilment of the requirements for:

CBS3011 – Usability Design of Software Applications

Submitted to:

Swarnalatha P.

Vellore Institute of Technology

Academic Year: 2026–2027

Abstract

Modern software development faces a growing challenge: static analysis tools generate large volumes of vulnerability findings, yet developers—particularly those with limited security expertise—lack the contextual guidance necessary to understand, prioritise, and remediate these issues effectively. This disconnect between automated detection and actionable developer knowledge represents a critical gap in contemporary software engineering practice.

Explicite AI addresses this gap by providing an integrated, AI-powered platform that combines static application security testing (SAST) with contextual AI-generated explanations, structured suggested fixes, and adaptive learning support. The platform enables developers to submit source code—either as a ZIP archive or via a public Git repository URL—and receive a comprehensive analysis covering security vulnerabilities, code quality issues, CVSS severity scores, and technical debt estimates.

The technical implementation employs a modern three-tier architecture: a React 19 and TypeScript frontend presenting an 'Explicite Midnight' dark-first design system; a Python 3.14 and FastAPI backend providing a RESTful API; and a PostgreSQL 16 database for persistent storage of analysis results, findings, and learning progress. Static analysis is performed by Semgrep 1.181.0, supporting Python, Java, JavaScript, and TypeScript. AI-powered explanations and suggested fixes are generated through a configurable provider layer supporting both locally hosted Ollama models (llama3:latest) and the Anthropic API.

Core implemented features include: asynchronous code analysis with job status polling; categorised vulnerability findings with CVSS 3.1 scoring; structured AI explanations covering what, why, how, remediation, and caveats; AI-generated code fixes with mandatory developer approval; an interactive multi-turn AI tutor session; personalised learning resource recommendations; and a configurable light/dark user interface.

The usability design applies principles from ISO 9241 and Nielsen's heuristics, emphasising clarity, discoverability, consistent visual hierarchy, and human oversight preservation. The platform includes 426 passing automated backend tests verified through a comprehensive Phase O live smoke test across 62 API checks.

Explicite AI contributes to three United Nations Sustainable Development Goals: SDG 4 (Quality Education) through its AI tutor and learning pathway; SDG 9 (Industry, Innovation and Infrastructure) through automated security analysis; and SDG 16 (Peace, Justice and Strong Institutions) through its contribution to secure and trustworthy digital systems. The platform is intended as an educational and development tool, empowering developers to move from passive vulnerability detection towards active understanding and effective remediation.

Table of Contents

(Update this field in Microsoft Word: right-click → Update Field)

List of Figures

Figure 1: Distribution of Implemented Project Modules

Figure 2: Backend Test Suite Results

Figure 3: Feature Implementation Status

Figure 4: Competitor Feature Comparison Matrix

Figure 5: Explicite AI SDG Contribution Mapping

Figure 6: Explicite AI Code Analysis Workflow

Figure 7: Technology Stack Maturity and Integration Depth

Figure 8: Login Screen (screenshot unavailable – application must be running)

Figure 9: Dashboard Screen (screenshot unavailable)

Figure 10: Issues Screen with Severity Classification (screenshot unavailable)

List of Tables

Table 1: Technology Stack Summary

Table 2: Literature Review Comparison

Table 3: Competitor Feature Comparison

Table 4: Implemented Functional Modules

Table 5: Database Entity Summary

Table 6: Backend Test Suite Results

Table 7: CVSS 3.1 Severity Ratings

Table 8: SDG Contribution Mapping

Table 9: Development Challenges and Solutions

Table 10: Team Contribution Statement

Table 11: API Endpoint Summary (Appendix A)

Table 12: Database Model Overview (Appendix B)

Table 13: Test Case Summary (Appendix E)

List of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
CVSS	Common Vulnerability Scoring System
CWE	Common Weakness Enumeration
DAST	Dynamic Application Security Testing
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
HTTP	Hypertext Transfer Protocol
IDE	Integrated Development Environment
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
ML	Machine Learning
NVD	National Vulnerability Database
ORM	Object-Relational Mapper
OWASP	Open Web Application Security Project
REST	Representational State Transfer
SAST	Static Application Security Testing
SDG	Sustainable Development Goal
SQL	Structured Query Language
SSRF	Server-Side Request Forgery
UI	User Interface
UX	User Experience
VIT	Vellore Institute of Technology
XAI	Explainable Artificial Intelligence

CHAPTER 1: Introduction

1.1 Background

Software development has evolved dramatically over the past two decades, moving from monolithic architectures to complex distributed systems comprising millions of lines of code. As software systems grow in scale and complexity, the challenge of ensuring their security and quality has become increasingly critical. According to the National Vulnerability Database, thousands of new software vulnerabilities are reported each year, many of which arise from well-understood, preventable coding mistakes such as SQL injection, path traversal, and improper input validation.

Traditional code review—in which developers manually inspect code for defects—remains a cornerstone of software quality assurance. However, manual review is resource-intensive, inconsistent across reviewers, and poorly suited to the volume and velocity of modern development workflows. A survey of software developers reported that developers spend an average of 6.4 hours per week on code review [2], representing a substantial cost for development teams of any size.

1.2 Static Application Security Testing

Static Application Security Testing (SAST) tools address these challenges by analysing source code automatically, without executing it, to identify potential vulnerabilities and code quality issues. Tools such as Semgrep, SonarQube, and GitHub CodeQL have demonstrated the ability to detect common vulnerability classes at scale. However, SAST tools present a fundamental usability challenge: their output consists of rule identifiers, file paths, and line numbers, with minimal contextual explanation of why a finding matters, how exploitable it is, or how a developer should respond to it.

Research has demonstrated that SAST tool performance varies significantly. Bennett et al. [6] found that individual SAST tools detected between 11.2% and 26.5% of known vulnerabilities in their evaluation dataset, with a combination of four tools reaching only 38.8% detection coverage. This highlights both the value and the limitations of rule-based static analysis as a standalone approach.

1.3 Role of Artificial Intelligence in Code Review

The emergence of large language models (LLMs) has created new possibilities for code analysis and review. Unlike rule-based static analysers, LLMs can understand the semantic context of code and generate natural-language explanations that are accessible to developers at all levels of expertise. Rasheed et al. [1] demonstrated that LLM-based agents can autonomously review code, identify bugs and code smells, and provide improvement

suggestions beyond what traditional static analysis tools offer. Google's AutoCommenter system [3], deployed to tens of thousands of developers, demonstrated that automated AI-assisted code review can have a measurable positive impact on developer workflows at industrial scale.

However, AI-generated explanations and suggested fixes introduce their own challenges: LLM outputs can be imprecise, incomplete, or incorrect, particularly for less common vulnerability types. Limbani et al. [4] found that while AI-assisted code review can improve precision and reduce review time, it may also introduce false positives and increase review duration if not carefully integrated. This reinforces the importance of explainable AI (XAI) principles—ensuring that AI-generated recommendations are accompanied by transparent reasoning that developers can evaluate.

1.4 Introduction to Explicite AI

Explicite AI is a full-stack web application developed to address the gap between automated vulnerability detection and developer-accessible, actionable remediation. The platform integrates static code analysis through Semgrep with AI-generated explanations, structured suggested fixes, and an interactive learning pathway, providing developers with a unified workflow from code submission to understanding and remediation.

The name "Explicite" reflects the platform's core design principle: making security findings explicit, understandable, and actionable rather than presenting them as opaque rule violations. Every finding in the system can be accompanied by a structured AI explanation covering what the vulnerability is, why it matters, how it arises in code, and how it can be remediated. Suggested fixes are proposed by the AI system but require explicit developer approval before any code modification is considered, preserving human oversight throughout the workflow.

1.5 Problem Statement

Developers lack an integrated tool that combines static-analysis reporting with contextual AI-generated explanations, structured remediation guidance, and adaptive learning support. This absence creates a workflow disconnect in which developers must navigate between analysis reports, documentation references, and issue trackers to understand and address vulnerabilities—creating friction, inconsistency, and missed learning opportunities, particularly for developers with limited security expertise.

1.6 Project Scope

Explicite AI is scoped as a locally deployable web application for individual developers and small development teams. The platform supports static analysis of Python, Java, JavaScript, and TypeScript source code submitted as ZIP archives or via public HTTPS Git repository URLs. AI features require either a locally hosted Ollama instance or access to the Anthropic

API. The platform does not support dynamic analysis, compiled binary analysis, private repository access, or production cloud deployment in its current form.

CHAPTER 2: Objectives and Project Scope

2.1 Primary Objectives

- Automate source-code security analysis for Python, Java, JavaScript, and TypeScript through Semgrep 1.181.0.
- Detect and classify potential vulnerabilities with severity ratings (Critical, High, Medium, Low).
- Present categorised security and quality findings through a structured, navigable interface.
- Explain vulnerabilities through AI-generated structured responses covering what, why, how, remediation, and caveats.
- Prioritise findings using a risk scoring system based on severity weights.
- Calculate and display CVSS 3.1 base scores where vector metadata is available from Semgrep rules.
- Generate AI-assisted suggested code fixes with explicit developer approval required before application.
- Provide curated educational resources and structured learning paths linked to vulnerability categories.
- Recommend learning material personalised to the user's open findings using a deterministic matching algorithm.
- Provide an interactive multi-turn AI learning tutor for contextual developer education.
- Apply a consistent, usability-focused "Explicite Midnight" dark-first design system across all 11 screens.

2.2 Secondary Objectives

- Reduce developer cognitive load by consolidating the analysis-to-remediation workflow in a single application.
- Provide an organised dashboard with aggregated project metrics including technical debt and duplication percentage.
- Maintain persistent analysis history enabling comparison of findings across multiple submissions.
- Support configurable AI providers (Ollama, Anthropic) through a single environment variable.
- Ensure a consistent user experience through design tokens, visual hierarchy, and clear feedback states.
- Implement role-based access control (viewer, developer, owner) for project membership.

2.3 Target Users

Explicite AI is designed for the following user groups:

Developers: Individuals who submit source code for analysis, inspect findings, and request AI explanations and suggested fixes.

Computer Science Students: Learners who use the AI tutor, learning resources, and structured explanations to develop secure coding knowledge.

Project Team Members: Participants who review analysis results across a shared project using the project membership system.

2.4 Functional Scope

The following features are implemented and verified in the current release:

- User registration, authentication (JWT HS256), and session management
- Project creation with role-based membership management
- ZIP archive upload with security validation (magic bytes, decompression bomb detection, path traversal prevention)
- Public HTTPS Git repository clone with SSRF protection (DNS validation, private IP blocking)
- Asynchronous Semgrep analysis with background job execution and status polling
- Vulnerability findings storage with severity, issue type, file path, and line number
- CVSS 3.1 base score computation from Semgrep rule metadata
- Technical debt estimation and code duplication detection
- AI-generated structured vulnerability explanations with adjustable expertise level
- AI assistant chat for project-scoped questions
- AI-generated suggested code fixes with approval/rejection workflow
- Learning resources, learning paths, and progress tracking
- Personalised learning recommendations based on open findings
- Multi-turn AI tutor sessions with persistent conversation history
- User settings including appearance (light/dark mode) and AI level preference

2.5 Out-of-Scope Functionality

- Dynamic Application Security Testing (DAST)
- Binary or bytecode analysis
- Private Git repository access
- Real-time IDE plugin integration
- Production cloud deployment or multi-tenancy
- CI/CD pipeline integration
- Automated patch application (all fixes require explicit developer approval)
- Support for languages beyond Python, Java, JavaScript, and TypeScript
- Formal user satisfaction studies or A/B testing

CHAPTER 3: Literature Review

3.1 Overview

This chapter reviews eight research papers directly relevant to Explicite AI's core areas: AI-powered code review, LLM-based code analysis, explainability in software engineering tools, static application security testing, deep learning for vulnerability detection, and automated program repair. Four papers were supplied with the project; four additional verified papers have been identified and reviewed.

3.2 Paper 1: AI-Powered Code Review with LLMs: Early Results

Rasheed, Z., Abdul Sami, M., Waseem, M., Kemell, K., Wang, X., Nguyen, A., Systä, K., and Abrahamsson, P. (2024). "AI-powered Code Review with LLMs: Early Results." arXiv preprint arXiv:2404.18496. Tampere University et al. [1]

This paper presents a multi-agent LLM-based system for autonomous code review. The system comprises four specialised agents: a Code Review Agent, Bug Report Agent, Code Smell Agent, and Code Optimisation Agent, each leveraging GPT-4 to perform distinct analysis tasks and communicate through a centralised coordination component. Unlike traditional static analysis, the proposed system can anticipate potential future risks in code and provide dual benefits: enhanced code quality and developer learning.

The study demonstrates that LLM-based agents can go beyond pattern matching to provide contextual feedback on best practices, coding standards, and code optimisation opportunities. A key finding is that the system supports developer learning by promoting understanding of effective coding techniques, not merely identifying issues. The authors identify the absence of LLM-specific tools for code review as a significant gap in existing research.

Limitations include the absence of an empirical evaluation with real developers, reliance on a proprietary model (GPT-4), and the experimental nature of the early results. The relevance to Explicite AI is direct: the paper supports the core design principle that AI explanations should promote developer understanding rather than simply surfacing findings.

3.3 Paper 2: Automated Code Review Using LLMs with Symbolic Reasoning

İçöz, B. and Biricik, G. (2025). "Automated Code Review Using Large Language Models with Symbolic Reasoning." IEEE, 979-8-3315-1482-2/25. Yıldız Technical University, Istanbul. [2]

This IEEE paper proposes a hybrid approach that integrates symbolic reasoning techniques with LLMs to automate the code review process. The study evaluates CodeT5, CodeBERT, and GraphCodeBERT on the CodexGlue Python defect detection dataset under four

conditions: base model, few-shot learning, fine-tuned model, and a hybrid approach combining symbolic knowledge maps with fine-tuned models.

Results show that GraphCodeBERT with the hybrid approach achieved the highest performance with accuracy of 68.7%, precision of 0.485, and F1-score of 0.381. The hybrid approach improved accuracy by up to 27.46% compared to the base model. The paper identifies a key limitation of LLMs: their tendency to produce semantically limited results and hallucinations in code review tasks. The symbolic reasoning component, implemented as a knowledge map embedded in prompts, provides additional context that guides models toward more logical findings.

Relevance to Explicite AI: this work informs the platform's approach of combining Semgrep's rule-based static analysis with LLM-generated explanations, creating a hybrid system that benefits from both the reliability of pattern-based tools and the contextual reasoning of language models.

3.4 Paper 3: AI-Assisted Assessment of Coding Practices (AutoCommenter)

Vijayvergiya, M., Salawa, M., Budiselić, I., Zheng, D., Lamblin, P., Ivanković, M., Carin, J., Lewko, M., Andonov, J., Petrović, G., Tarlow, D., Maniatis, P., and Just, R. (2024). "AI-Assisted Assessment of Coding Practices in Modern Code Review." AIware '24, July 15–16, 2024, Porto de Galinhas, Brazil. ACM. [3]

This paper reports on the development, deployment, and evaluation of AutoCommenter, a large-scale LLM-based code review system deployed to tens of thousands of developers at Google. AutoCommenter automatically learns and enforces coding best practices for C++, Java, Python, and Go by detecting violations and generating automated code review comments.

The system employs a T5-based large sequence model trained on over 3 billion examples, fine-tuned on historical code review comment data. The model detects violations by predicting the code location and the URL of the violated best practice. Evaluation showed a positive impact on developer workflow and confirmed the feasibility of end-to-end automated best-practice enforcement at industrial scale. Key lessons include the importance of calibrating confidence thresholds to maintain acceptable false positive rates and the engineering challenges of deploying such a system at scale.

Limitations include reliance on a large proprietary infrastructure, computational cost of continuous model retraining, and the challenge of managing an ever-evolving set of best practice guidelines. Relevance: AutoCommenter's architecture validates Explicite AI's design of a backend AI service layer with a swappable provider abstraction, and its lessons on confidence thresholds inform the platform's handling of AI output validation.

3.5 Paper 4: AI-Assisted Code Reviews on Developer Productivity

Limbani, R.Y., Adityakumar, P., and Basit, S.M. (2025). "Evaluating The Impact Of AI-Assisted Code Reviews On Developer Productivity And Code Quality." International Journal of Creative Research Thoughts (IJCRT), Volume 13, Issue 4, April 2025, ISSN: 2320-2882. Article ID: IJCRT25A4167. [4]

This paper evaluates how AI-assisted code review platforms affect developer productivity and code quality by integrating GPT-4, CodeBERT, and CodeT5 with traditional static analysis tools (Pylint, ESLint). The study presents an AI platform evaluated on benchmark code snippets containing known issues, measuring review time, Acc@k, Precision@1, and Average Precision against baseline automated systems.

Results show that the AI platform achieved Precision@1 of 95% versus the baseline, and reduced review time from 15 seconds to 10 seconds per snippet. However, Acc@k (86%) was slightly lower than the baseline (93%), suggesting that while the AI platform provides more precise and actionable feedback, it may occasionally miss exact fault locations. A central finding is that Explainable AI (EAI) principles—providing natural-language justifications for findings—are critical for developer trust and adoption. Without explanations, even accurate AI outputs may be ignored or mistrusted.

Limitations include a small benchmark dataset and the fact that results may not generalise to complex real-world codebases. Relevance: this paper directly supports Explicite AI's structured five-field explanation format (what, why, how, remediation, caveats) as a concrete implementation of EAI principles.

3.6 Paper 5: VulDeePecker – Deep Learning for Vulnerability Detection

Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., and Zhong, Y. (2018). "VulDeePecker: A Deep Learning-Based System for Vulnerability Detection." Proceedings of the 25th Annual Network and Distributed System Security Symposium (NDSS 2018), San Diego, CA, 18–21 February 2018. DOI: 10.14722/ndss.2018.23158. [5]

VulDeePecker is a pioneering deep learning system for automated vulnerability detection in C/C++ source code. The system introduces the concept of "code gadgets"—groups of semantically related lines of code that need not be consecutive—as the fundamental input unit for a bidirectional LSTM network. The authors also contribute the first vulnerability dataset designed specifically for deep learning approaches.

Evaluated on CWE-119 (buffer error) and CWE-399 (resource management error), VulDeePecker achieved significantly fewer false negatives than traditional tools while maintaining reasonable false positive rates. Applied to real-world software (Xen, Seamonkey,

Libav), the system detected four previously unreported vulnerabilities that had been silently patched by vendors without NVD disclosure, demonstrating its practical detection capability.

Limitations include restricted language support (C/C++ only), focus on specific CWE classes, and the requirement for labelled training data that may not cover emerging vulnerability types. Relevance to Explicite AI: VulDeePecker's work on code gadgets and semantic feature extraction informs the conceptual foundation of relating vulnerability detection to meaningful code units—a principle reflected in Explicite AI's per-finding file path and line number attribution.

3.7 Paper 6: Semgrep* – Improving SAST Tool Performance

Bennett, G., Hall, T., Winter, E., and Counsell, S. (2024). "Semgrep*: Improving the Limited Performance of Static Application Security Testing (SAST) Tools." Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024 / SECUTE Workshop), Salerno, Italy. ACM, pp. 614–623. DOI: 10.1145/3661167.3661262. [6]

This paper directly evaluates the performance limitations of SAST tools, including Semgrep, on production code with known vulnerabilities. Using a manually curated dataset, the authors found that individual SAST tools detected only 11.2%–26.5% of vulnerabilities. Combining four tools improved coverage to 38.8%. The authors then authored custom Semgrep rules targeting detected gaps, producing a tuned version ("Semgrep*") that detected 44.7% of vulnerabilities—a 181% improvement over the original Semgrep detection rate.

The study also describes a containerised DevSecOps pipeline for injecting known vulnerabilities and benchmarking SAST tools systematically. A key finding is that the performance of rule-based tools is highly dependent on the quality and coverage of their rule sets, and that domain-specific rule authoring can substantially improve detection.

Relevance to Explicite AI: this paper provides direct evidence supporting the platform's design rationale for AI-augmented analysis. When Semgrep alone detects fewer than half of known vulnerabilities, AI-generated contextual explanations and developer education become important tools for filling the gap between automated detection and comprehensive security awareness.

3.8 Paper 7: Automated Repair of Programs from LLMs

Fan, Z., Gao, X., Mirchev, M., Roychoudhury, A., and Tan, S.H. (2023). "Automated Repair of Programs from Large Language Models." Proceedings of the 45th IEEE/ACM International Conference on Software Engineering (ICSE 2023), Melbourne, Australia. IEEE/ACM, pp. 1469–1481. DOI: 10.1109/ICSE48619.2023.00128. [7]

This ICSE 2023 paper systematically studies whether automated program repair (APR) techniques can fix incorrect solutions generated by language models in competitive programming contexts (LeetCode). The authors evaluate whether Codex (edit mode) and conventional Java APR tools (TBar, Recoder) can automatically repair LLM-produced code with fault location information.

Results show that Codex's edit mode performs comparably to or better than the Java repair tools when given fault location information. The paper demonstrates that automated repair of LLM-generated code is feasible but that the quality of the repair depends heavily on the accuracy of fault localisation. This finding is directly applicable to the challenge of validating AI-generated suggested fixes.

Limitations include the constrained evaluation context (competitive programming problems with automated test cases) which may not generalise to complex, real-world security vulnerabilities. Relevance to Explicite AI: this paper supports the platform's fix workflow design, where proposed fixes require developer review and approval rather than automated application—acknowledging that LLM-generated fixes have meaningful error rates.

3.9 Paper 8: Towards Explainable Vulnerability Detection with LLMs

Mao, Q., Li, Z., Hu, X., Liu, K., Xia, X., and Sun, J. (2024). "Towards Explainable Vulnerability Detection with Large Language Models." arXiv preprint arXiv:2406.09701, Zhejiang University / York University. Submitted June 2024. [8]

This paper proposes LLMVulExp, a teacher-student framework in which a "teacher" LLM generates structured explanation data for vulnerability findings, which is then used to train a "student" model capable of both detecting and explaining vulnerabilities. The framework introduces an evaluation protocol for AI-generated explanations using three metrics: Accuracy, Clarity, and Actionability, assessed through an LLM-as-judge approach.

Evaluated on the SeVC dataset, LLMVulExp achieved over 90% F1 score on vulnerability detection while generating explanations rated significantly higher on all three quality metrics than baseline approaches. The paper demonstrates that structured explanations—not merely vulnerability labels—are achievable through LLM-based approaches and that the quality of explanations can be systematically measured.

Limitations include the preprint status (no peer-reviewed venue confirmed at time of writing), dataset scope, and the resource requirements for the teacher-student training process. Relevance to Explicite AI: LLMVulExp's five-dimension explanation framework (analogous to Explicite AI's what/why/how/remediation/caveats structure) and its emphasis on actionable explanations directly validates the platform's structured explanation design.

3.10 Literature Comparison

The following table summarises the eight reviewed papers across key dimensions relevant to Explicite AI:

#	Authors (Year)	Research Area	Key Contribution	Limitations	Relevance
[1]	Rasheed et al. (2024)	LLM Code Review	Multi-agent GPT-4 review	No empirical eval	AI explanation design
[2]	İçöz & Biricik (2025)	Hybrid Code Review	Symbolic+LLM hybrid	Limited dataset	Hybrid analysis rationale
[3]	Vijayvergiya et al. (2024)	Industrial AI Review	AutoCommenter at Google scale	Proprietary infrastructure	Provider abstraction design
[4]	Limbani et al. (2025)	AI Review Impact	EAI for developer trust	Small benchmark	Structured explanations
[5]	Li et al. (2018)	Vuln Detection ML	Code gadgets + LSTM	C/C++ only	Per-finding attribution
[6]	Bennett et al. (2024)	SAST Evaluation	Semgrep* rule authoring	Curated dataset	AI augmentation rationale
[7]	Fan et al. (2023)	Program Repair	APR for LLM code	Constrained eval	Fix approval workflow
[8]	Mao et al. (2024)	XAI + Vuln Detection	Teacher-student XAI	Preprint only	Explanation structure

3.11 Research Gap Analysis

The reviewed literature reveals several consistent gaps that Explicite AI is designed to address:

Absence of integrated explanation-analysis tools: Existing SAST tools detect vulnerabilities but do not explain them in developer-accessible terms. LLM explanation systems exist but are not integrated with static analysis pipelines.

Separation between detection and remediation: Static analysers report findings; developers must separately seek remediation guidance. No single tool closes this loop from detection to explanation to fix to learning.

Limited support for developer education: Commercial SAST tools are not designed as educational platforms. VulDeePecker and LLMVulExp focus on detection accuracy rather than developer learning outcomes.

Difficulty prioritising findings: Without risk scores, severity ratings, and CVSS scores presented together, developers cannot effectively prioritise which findings to address first.

Accessibility for less experienced developers: SAST outputs are designed for security experts. Adjustable explanation levels and interactive tutor sessions, as implemented in Explicite AI, are not features of existing tools.

CHAPTER 4: Existing Systems and Competitor Analysis

4.1 Overview of Existing Approaches

The automated code security analysis market encompasses a range of tools from open-source rule engines to enterprise SAST platforms. Each tool represents a different trade-off between analysis depth, language support, deployment model, and developer-facing features. This chapter reviews five established tools and positions Explicite AI relative to them.

4.2 SonarQube

SonarQube is a widely deployed open-source (Community Edition) and commercial static analysis platform supporting over 30 programming languages. It performs continuous code quality and security analysis, detecting bugs, code smells, and security hotspots. SonarQube integrates with CI/CD pipelines and provides a web dashboard with quality gates, historical trend analysis, and issue management. The commercial editions include AI-generated suggestions and extended security rule sets. SonarQube does not natively provide structured natural-language explanations of individual findings or developer learning pathways.

4.3 Snyk Code

Snyk Code is a commercial SAST tool that combines semantic code analysis with AI-powered fix suggestions. It supports real-time IDE integration (VS Code, IntelliJ), pull request analysis, and CI/CD pipeline integration. Snyk Code provides fix recommendations and natural-language descriptions for findings, making it one of the closer commercial analogues to Explicite AI's explanation approach. However, it does not provide structured multi-field explanations covering why a vulnerability matters and how it arises, nor does it provide an educational tutor or learning pathway integrated into the analysis workflow.

4.4 Semgrep

Semgrep is an open-source, pattern-based static analysis engine that forms the analysis backbone of Explicite AI. As a standalone tool, Semgrep executes rule sets against source code and produces findings with severity, rule ID, file path, and line number. Semgrep OSS does not provide AI-generated explanations, risk prioritisation beyond rule severity, or suggested fixes. Semgrep Code (commercial) adds some AI-powered features. The open-source version, as used in Explicite AI, is entirely dependent on rule quality and provides no contextual developer guidance beyond the rule description string.

4.5 GitHub CodeQL

CodeQL is GitHub's semantic code analysis engine that models source code as a queryable database, enabling complex data-flow and control-flow queries. CodeQL is highly effective

for detecting complex vulnerability classes that require cross-function data flow analysis. It is integrated into GitHub Advanced Security and supports C, C++, C#, Java, JavaScript, Python, Ruby, and Swift. CodeQL does not provide developer-facing AI explanations of individual findings, suggested fixes, or learning resources in its standard integration.

4.6 Checkmarx

Checkmarx is a commercial enterprise SAST platform offering broad language support, CI/CD integration, and compliance reporting. The Checkmarx One platform includes AI-powered features such as AI-generated remediation guidance and correlation with open-source dependency risks. It is designed for enterprise security teams rather than individual developers or students, and its pricing model reflects this positioning. Checkmarx provides remediation guidance but does not offer an integrated learning pathway or interactive tutor.

4.7 Competitor Comparison Matrix

Table 3 compares the six tools across key dimensions. Feature availability is assessed based on official documentation and publicly available information as of 2025–2026. "Partial" indicates limited or commercial-tier-only availability.

Feature	SonarQube	Snyk Code	Semgrep	CodeQL	Checkmarx	Explicite AI
Static Analysis	✓	✓	✓	✓	✓	✓
Multi-language Support	✓	✓	✓	✓	✓	Partial (4)
AI Explanations	~	~	✗	✗	~	✓
Structured Explanation	✗	✗	✗	✗	✗	✓
Risk / CVSS Scoring	✓	✓	~	~	✓	✓
Suggested Fixes	~	✓	✗	✗	~	✓ (advisory)
Developer Learning	✗	✗	✗	✗	✗	✓
Interactive AI Tutor	✗	✗	✗	✗	✗	✓
CI/CD Integration	✓	✓	✓	✓	✓	✗ (planned)

Open Source / Free Tier	✓	~	✓	✓	✗	✓
Git Repo Analysis	~	✓	~	✓	✓	✓

Note: ✓ = Supported; ~ = Partial/Commercial Tier; ✗ = Not Available. Competitor feature assessment based on official documentation (2025–2026). Explicite AI's suggested fixes are advisory only and require explicit developer approval.

4.8 Positioning and Differentiation

Explicite AI occupies a distinct position in this landscape: it is the only tool in the comparison that integrates structured AI explanations, advisory suggested fixes, and an interactive developer learning pathway in a single open application. Its differentiation is not in analysis depth—commercial tools such as Checkmarx and CodeQL offer more comprehensive analysis capabilities—but in its focus on making vulnerability findings understandable, actionable, and educational for developers at all experience levels.

Importantly, Explicite AI does not claim to outperform commercial SAST platforms in detection accuracy. Its intended use case is as an educational and developer-assistance platform that augments rather than replaces specialised security tooling. The advisory nature of its AI-generated fixes, combined with mandatory developer approval, reflects a design philosophy of human oversight preservation rather than fully automated remediation.

CHAPTER 5: Proposed System and System Architecture

5.1 System Overview

Explicite AI is implemented as a three-tier web application comprising a React frontend, a FastAPI backend, and a PostgreSQL database, with external integrations to the Semgrep analysis engine and a configurable AI provider layer. All communication between tiers occurs via HTTPS/JSON REST API calls. The system is designed for local deployment with clear separation of concerns between the user interface, business logic, data persistence, and external analysis services.

5.2 System Architecture

The three-tier architecture is structured as follows:

Browser / Frontend (port 8443): The React 19 and TypeScript frontend communicates with the backend exclusively through the api.ts HTTP client module. Navigation between 11 screens is managed by a custom NavCtx React context, eliminating the need for a routing library. JWT tokens are stored in sessionStorage under the key "explicite_token". The Explicite Midnight design system is implemented through Tailwind CSS v4 with --xm-* CSS custom properties supporting light and dark theme modes via a data-theme HTML attribute.

Backend / API Layer (port 8000): The FastAPI application provides RESTful endpoints under the /api/v1 prefix. Authentication uses JWT HS256 bearer tokens with configurable expiry. Role-based access control is enforced at the dependency injection layer (deps.py), where the get_project_membership() function validates the current user's membership and role before any project-scoped operation proceeds. Analysis jobs execute asynchronously as background tasks, returning HTTP 202 immediately and storing results in the database upon completion.

Persistence Layer (port 5434): PostgreSQL 16 running in a Docker container stores all application data. SQLAlchemy 2.x async ORM with the asyncpg driver handles all database interactions. Alembic manages the schema migration chain with nine migrations from the initial schema to the current head (c1d2e3f4a5b6).

External Services: Semgrep 1.181.0 executes as a subprocess within the backend's virtual environment. AI services (Ollama at localhost:11434 or the Anthropic API) are accessed through a provider abstraction layer in ai_service.py, switchable via the LLM_PROVIDER environment variable.

5.3 Data Flow

The primary analysis workflow follows an asynchronous pipeline:

1. Developer authenticates via POST /api/v1/auth/login; receives a JWT bearer token.

2. Developer selects or creates a project; project membership is verified server-side.

3. Developer submits code via POST /api/v1/projects/{id}/analyse (ZIP) or /analyse/git (Git URL).

4. Backend creates an AnalysisJob record with status "queued" and returns HTTP 202.

5. Background task executes: source is extracted/cloned, Semgrep analyses the source tree.

6. Findings, metrics (technical debt, duplication), and CVSS scores are stored in the database.

7. Frontend polls GET /api/v1/projects/{id}/analyse/{job_id} until status is "completed" or "failed".

8. Developer inspects findings, requests AI explanations, generates suggested fixes, and accesses learning resources.

Figure 6: Explicite AI Code Analysis Workflow (Source: Project implementation)

5.4 Security Architecture

Security controls are implemented at multiple layers:

Input validation: Pydantic v2 schemas validate all request bodies; ZIP archives are checked for magic bytes, decompression bomb ratios (>100:1), and entry count limits (max 1,000).

SSRF protection: Git repository URLs must use HTTPS. The git_service.py module performs DNS resolution and validates all resolved IP addresses against private ranges (RFC 1918, loopback, link-local) before initiating a clone operation.

Safe process execution: Semgrep is invoked with a fixed argument list using shell=False, preventing injection via user-controlled file paths.

Source isolation: Extracted source is processed in temporary directories. source_path is never included in any API response. The .git/ directory is removed after clone.

Fix safety: No fix is applied without explicit developer approval. Stale-source detection compares a SHA-256 hash of the original file at approval time against the hash recorded at fix generation time.

AI output handling: Finding data is labelled as DATA in LLM prompts as an injection mitigation. Model output is validated for required fields and length limits before storage.

5.5 Database Entity Relationship

The database schema comprises eight primary entities with the following relationships:

Entity	Description
users	Stores user credentials (hashed), preferences, and system role. One-to-many with project memberships.
projects	Stores project metadata. Associated with users through the project_memberships join entity.
project_memberships	Join entity linking users to projects with role assignment (viewer/developer/owner).
analysis_jobs	Stores analysis job state, status, source metadata, repo_url (nullable), metrics, and timestamps. One-to-many with findings.
findings	Stores individual vulnerability findings with severity, issue type, file path, line numbers, rule key, CVSS vector, and status.
suggested_fixes	Stores AI-generated fix proposals including proposed_lines, rationale, caveats, original_hash, and approval status.
learning_resources / learning_paths	Stores educational content and structured learning sequences. Many-to-many through learning_path_resources.
tutor_sessions / tutor_messages	Stores multi-turn AI tutor conversations with resource context and message history.

5.6 AI Explanation Architecture

The AI service layer (ai_service.py) implements a provider abstraction pattern. The AIProvider abstract base class defines the interface: generate_explanation(), generate_fix(), and chat(). Concrete implementations are OllamaProvider and AnthropicProvider, both implementing identical method signatures. A NoneProvider returns honest 503 errors when no AI is configured. The active provider is selected at application startup based on LLM_PROVIDER and requires no API or frontend changes to switch between providers.

System prompts explicitly label finding data as DATA rather than instructions, as a prompt injection mitigation measure. Explanation responses are validated for all five required JSON fields (what, why, how_it_arises, remediation, caveats) and each field is truncated at a maximum length before storage. Fix generation uses LLM_FIX_MAX_TOKENS=2048 (configurable) to accommodate the larger token output required for proposed code replacements.

CHAPTER 6: Technology Stack

6.1 Overview

Explicite AI employs a deliberately curated technology stack selected for developer productivity, async performance, and ecosystem maturity. Each technology choice reflects specific requirements of the platform: long-running analysis jobs demanded an async backend; real-time job status polling required a reactive frontend; security analysis required a programmable, subprocess-safe analysis engine.

6.2 Frontend Technologies

React 19: Component-based UI framework. Selected for its hooks-based state management, concurrent rendering capabilities, and wide developer adoption. The application uses React 19's latest stable API including React Context for global navigation state (NavCtx).

TypeScript: Statically typed superset of JavaScript. Provides compile-time type checking across all frontend components, API response types, and navigation state. Eliminates a class of runtime type errors in API communication.

Vite 8: Build tool and development server. Provides hot module replacement (HMR) for rapid development feedback and produces optimised production bundles. Frontend development server runs on port 8443.

Tailwind CSS v4: Utility-first CSS framework. Combined with --xm-* CSS custom properties to implement the Explicite Midnight design system. Theme switching uses the data-theme HTML attribute to override the full CSS variable set.

Inter + JetBrains Mono: Typography: Inter (Google Fonts) for UI text; JetBrains Mono for code display, terminal output, and technical values throughout the interface.

6.3 Backend Technologies

Python 3.14: Primary backend language. Selected for its extensive library ecosystem (FastAPI, SQLAlchemy, Pydantic, Semgrep) and native async/await support enabling non-blocking I/O throughout the API layer.

FastAPI: Async web framework built on Starlette and Pydantic. Provides automatic OpenAPI documentation, native async route handlers, dependency injection, and tight integration with Pydantic for request validation. All API routes use async def.

Pydantic v2: Data validation library. Defines all request and response schemas, enforcing type validation at system boundaries. Used for settings management (pydantic-settings) and Alembic configuration.

SQLAlchemy 2.x (async): ORM with full async support via AsyncSession and asyncpg. All database operations are non-blocking. Declarative models define the schema; queries use the 2.x Select API.

asyncpg: High-performance async PostgreSQL driver. Used as the DBAPI backend for SQLAlchemy async sessions.

Alembic: Database migration framework. Manages the nine-migration chain from initial schema to current head (c1d2e3f4a5b6). Migrations are authored as Python scripts with async support.

6.4 Database

PostgreSQL 16 is deployed via Docker, listening on host port 5434. PostgreSQL was selected for its robustness, JSONB support, and excellent compatibility with SQLAlchemy async. A separate test database (test_explicite_ai) is used for the automated test suite, ensuring that tests never touch production data. The Alembic migration chain provides reproducible schema management and tracks all structural changes to the database.

6.5 Security Analysis Engine

Semgrep 1.181.0 is the static analysis engine. It supports Python, Java, JavaScript, and TypeScript rule sets from the Semgrep Registry (r/ prefix). Semgrep is invoked as a subprocess within the backend virtual environment with a fixed argument list (shell=False). The source directory path is passed as an argument, not interpolated into a shell string, preventing injection. Semgrep outputs JSON which is parsed by the analysis_engine.py adapter into structured Finding records. CVSS vector strings extracted from Semgrep rule metadata are scored using a standards-compliant CVSS 3.1 implementation in cvss_scoring.py.

6.6 Artificial Intelligence Layer

The AI service supports two active providers and one null provider:

Ollama (llama3:latest): Locally hosted inference server at localhost:11434. llama3:latest (8B Q4_0 quantised) is the default active model. No API key required. LLM_MAX_TOKENS=512 for explanations and chat; LLM_FIX_MAX_TOKENS=2048 for fix generation. Timeout: LLM_TIMEOUT_SECONDS=30.

Anthropic API: Optional cloud provider using claude-haiku-4-5. Requires ANTHROPIC_API_KEY. Switchable via LLM_PROVIDER=anthropic without any code changes.

None Provider (default): When LLM_PROVIDER=none (the default in .env.example), all AI endpoints return HTTP 503 with an honest "AI provider not configured" message. No silent fallback or fabricated response is returned.

6.7 Technology Stack Summary

Layer	Technology	Version	Purpose
Frontend	React + TypeScript	19 / 5.x	Component UI, state management
Frontend	Vite	8	Build tool, dev server (8443)
Frontend	Tailwind CSS	v4	Utility styling + design system
Backend	Python	3.14	Primary backend language
Backend	FastAPI	Latest	Async REST API framework
Backend	Pydantic	v2	Validation, settings, schemas
Database	PostgreSQL	16	Relational data store (5434)
Database	SQLAlchemy async	2.x	ORM for async DB access
Database	Alembic	Latest	Schema migrations (9 total)
Analysis	Semgrep	1.181.0	Static analysis engine
Analysis	CVSS scoring	3.1	Vulnerability severity scoring
AI	Ollama	Latest	Local LLM inference server
AI	llama3:latest	8B Q4_0	Active language model
AI	Anthropic API	Optional	Cloud AI provider
Testing	pytest + asyncio	Latest	Backend test suite (426 tests)
Testing	httpx	Latest	Async HTTP test client



Figure 7: Technology Stack Integration Depth (Source: Project implementation)

CHAPTER 7: Functional Features and Application Modules

7.1 User Authentication

Authentication is implemented using JWT HS256 bearer tokens. Users register with an email and password; passwords are hashed using bcrypt before storage. The POST /api/v1/auth/login endpoint validates credentials and issues a signed JWT with a configurable expiry (JWT_ACCESS_TOKEN_EXPIRE_MINUTES). The token is stored in sessionStorage under the key "explicite_token" and included as a Bearer header in all subsequent API requests. The GET /api/v1/auth/me endpoint returns the current user's profile without exposing password hashes. PATCH /api/v1/auth/me allows users to update preferences and AI explanation level.

7.2 Project Management

Projects serve as the top-level container for analysis jobs and findings. A user creates a project via POST /api/v1/projects and becomes its owner. Other users can be added as members with viewer, developer, or owner roles. Every project-scoped API endpoint validates the current user's membership before proceeding. System administrators bypass membership checks. The project dashboard (GET /projects/{id}/dashboard) aggregates metrics including total findings, technical debt, duplications percentage, and risk score across the project's analysis history.

7.3 Source Code Analysis

7.3.1 ZIP Archive Analysis

Developers upload source code as a ZIP archive (max 10 MB) to POST /api/v1/projects/{id}/analyse. The archive.py module validates the submission: it checks for the ZIP magic bytes (PK\x03\x04), verifies that no individual entry exceeds a 100:1 compression ratio (decompression bomb detection), enforces a 1,000-entry count limit, prevents path traversal by resolving all paths within the extraction directory, and enforces a 50 MB uncompressed size limit. Valid archives are extracted to a temporary directory for Semgrep analysis.

7.3.2 Git Repository Analysis

Developers submit a public HTTPS Git repository URL and branch name to POST /api/v1/projects/{id}/analyse/git. The git_service.py module performs multi-stage validation: the URL must use HTTPS (not HTTP, SSH, file://, or git://); embedded credentials in the URL are rejected; the branch name is validated against an alphanumeric allowlist regex; DNS resolution is performed on the hostname; all resolved IP addresses are checked against private ranges (RFC 1918, loopback 127.0.0.0/8, and link-local 169.254.0.0/16). The clone

uses git clone --depth 1 with GIT_TERMINAL_PROMPT=0 and enforces size and file-count limits. The .git/ directory is removed after cloning to prevent accidental exposure of repository metadata. The commit SHA is recorded with the analysis job.

7.3.3 Asynchronous Job Lifecycle

Both analysis paths return HTTP 202 immediately upon job creation, with the job in "queued" status. The background task transitions the job through "running" to "completed" or "failed". Failure reasons are sanitised before storage to remove temporary source paths. The frontend polls GET /api/v1/projects/{id}/analyse/{job_id} until the job reaches a terminal state. Job metrics recorded on completion include: findings count, technical debt in minutes (configurable per finding type), duplication percentage (hash-based block detection), and a project-level risk score (severity-weighted).

7.4 Vulnerability Findings

Each finding stores: severity (Critical/High/Medium/Low), issue_type (Vulnerability/Maintainability/CodeSmell/other), file path, start/end line and column, rule key, description, CVSS vector (if available), CVSS base score, CVSS version, and status (open/acknowledged/resolved). The issues list endpoint supports filtering by severity, issue type, status, and a keyword search query. CVSS 3.1 scores are computed from vector strings using a standards-compliant implementation; CVSS 4.0 vectors are stored but not scored with the v3.1 formula. Findings without CVSS data (general code quality findings) display no score rather than a fabricated zero.

7.5 AI Vulnerability Explanations

The POST /api/v1/projects/{id}/issues/{iid}/explain endpoint accepts an explanation level parameter (beginner/intermediate/advanced) and returns a structured five-field explanation: what (the finding in plain terms), why (the security or reliability risk), how_it_arises (how this pattern appears in code), remediation (concrete steps to address it), and caveats (limitations, false-positive risk, missing context). Each field is truncated at 600 characters. The explanation level adjusts the vocabulary and assumed knowledge in the system prompt. AI responses are validated for all five required fields before storage; a malformed response triggers AiInvalidResponseError (HTTP 502).

7.6 AI Assistant

The POST /api/v1/projects/{id}/assistant endpoint provides a general-purpose AI chat scoped to the selected project. The system prompt includes the project name as context. Developers can ask questions about findings, analysis results, or security concepts without needing to navigate to a specific issue. The assistant uses LLM_MAX_TOKENS=512 for responses.

7.7 Suggested Fix Generation and Workflow

The fix workflow consists of five stages:

Generate: POST .../issues/{iid}/fix sends finding context to the AI, requesting replacement lines, rationale, and caveats. Uses LLM_FIX_MAX_TOKENS=2048 to accommodate longer code output.

Review: The proposed fix is stored with the original source hash (SHA-256 of the full file content). The developer reviews the unified diff, rationale, and caveats in the frontend.

Stale check: At approval time, the current source file's hash is recomputed and compared against the stored original_hash. A mismatch (409) prevents approving a fix generated against an outdated source version.

Approve: POST .../fixes/{fid}/approve requires developer role. A developer-approved fix is not automatically applied to any file.

Download: GET .../fixes/{fid}/download returns an in-memory ZIP containing the patched source file. The patch is constructed in memory; original source files are never modified.

7.8 Learning and Tutor System

The learning module provides three interconnected capabilities:

Learning resources: Curated articles and guides accessible via GET /api/v1/learning/resources. Resources are tagged by vulnerability category, CWE, and OWASP mapping.

Learning paths: Structured sequences of resources with progress tracking. Developers can mark resources as completed via POST /learning/resources/{id}/complete. Progress is tracked per-user.

Personalised recommendations: A deterministic matching algorithm scores resources against the user's open findings: rule_key match scores 3 points, issue_type match scores 2, tag match scores 1. Completed resources are excluded. Results are weighted by the finding's risk score.

Interactive AI tutor: POST /learning/tutor/start creates a per-user session linked to a specific learning resource. Subsequent POST /learning/tutor/{session_id}/chat messages maintain conversation history, with the resource topic injected into the AI system prompt. Sessions persist across browser sessions.

7.9 User Settings

The settings screen allows users to update their profile, change their preferred AI explanation level (beginner/intermediate/advanced), and toggle the application theme between Explicite Midnight (dark) and Light mode. Settings are persisted in the users table via PATCH /api/v1/auth/me. The 422 validation error handler in Settings.tsx uses the structured ApiError.detail payload to display field-level error messages directly alongside the relevant form field.

7.10 Functional Module Summary

Module	Backend Endpoint(s)	Frontend Screen	Status
Authentication	POST /auth/login, GET/PATCH /auth/me	Login, Settings	Implemented
Project Management	POST/GET /projects, GET /dashboard	Projects, Dashboard	Implemented
ZIP Analysis	POST /analyse	Code Analysis	Implemented
Git Analysis	POST /analyse/git	Code Analysis	Implemented
Job Status	GET /analyse/{id}	Code Analysis	Implemented
Findings List	GET /issues	Issues	Implemented
Finding Detail	GET /issues/{id}	Issue Detail	Implemented
CVSS Scoring	Computed server-side	Issue Detail	Implemented
AI Explanation	POST /issues/{id}/explain	Issue Detail	Implemented
AI Assistant	POST /assistant	AI Assistant	Implemented
Suggested Fix	POST /issues/{id}/fix	Suggested Fix	Implemented
Fix Approve/Reject	POST /fixes/{id}/approve|reject	Suggested Fix	Implemented
Fix Download	GET /fixes/{id}/download	Suggested Fix	Implemented
Learning Resources	GET /learning/resources	Learning	Implemented
Learning Paths	GET /learning/paths	Learning	Implemented
Recommendations	GET /learning/recommendations	Learning	Implemented
Tutor Session	POST /tutor/start, /chat	Tutor Session	Implemented
Settings	PATCH /auth/me	Settings	Implemented

Figure 1: Distribution of Implemented Project Modules (Source: Project implementation)

CHAPTER 8: User Interface and Usability Design

8.1 User-Centred Design Principles

Explicite AI's user interface was designed in accordance with ISO 9241-110 usability principles and Nielsen's ten usability heuristics. The target user base—developers and computer science students—was kept in mind throughout every design decision. The primary usability goals were: clarity (finding what matters quickly), discoverability (knowing where to go next), consistency (feeling at home on every screen), and feedback (always knowing what the system is doing).

8.2 Explicite Midnight Design System

All visual design is governed by the "Explicite Midnight" design system, a dark-first theme implemented through CSS custom properties with the prefix --xm-. This approach ensures that every screen shares identical colour semantics, eliminating visual inconsistency and reducing cognitive load during extended use. The full token set is overridden for light mode through the html[data-theme="light"] selector, providing a complete alternative theme without duplicating component styles.

8.3 Design Tokens

Token	Dark Value	Light Value	Usage
--xm-bg-app	#090B10	#F6F8FA	Page background
--xm-bg-sidebar	#0D1117	#FFFFFF	Navigation sidebar
--xm-bg-surface	#11151D	#FFFFFF	Cards and panels
--xm-bg-elevated	#1B2330	#F0F3F6	Elevated / modal surfaces
--xm-accent	#4F8EF7	#0969DA	Primary interactive elements
--xm-text	#E6EDF3	#1C2128	Primary text
--xm-severity-critical	Red	Red	Critical severity badges
--xm-severity-high	Amber	Amber	High severity badges
--xm-severity-medium	Cyan	Cyan	Medium severity badges
--xm-severity-low	Green	Green	Low severity badges

8.4 Typography

Body and interface text uses Inter (Google Fonts, variable weight 300–700, optical size 14–32px), selected for its legibility at small sizes and its clean, professional appearance in developer tooling. All code snippets, file paths, terminal output, and technical values are displayed in JetBrains Mono (400/500 weight), a monospace typeface optimised for code readability. Font loading uses Google Fonts with display=swap for performance.

8.5 Screen Inventory and Navigation

The application comprises 11 screens navigated through a persistent left sidebar. Navigation is managed by the NavCtx React context using a useState<Screen> hook, eliminating the need for a router library and simplifying the global navigation model. Screen transitions are immediate with no loading state between screens.

Screen	Primary Purpose
Login	Entry point. Email/password authentication. JWT stored in sessionStorage on success.
Dashboard	Aggregated metrics for the selected project: findings by severity, technical debt, duplication %, risk score.
Projects	Project list and creation. Displays member count, finding count, last analysis date.
Code Analysis	Source submission (ZIP or Git URL), job status display, and analysis history.
Issues	Paginated, filterable findings list with severity badges, issue type, and file path.
Issue Detail	Full finding detail including CVSS score, AI explanation accordion, and fix generation controls.
AI Assistant	Project-scoped AI chat for developer questions.
Suggested Fix	Diff viewer, rationale panel, caveats, and approval/rejection controls.
Learning	Resource library, learning paths, progress tracking, and personalised recommendations.
Tutor Session	Multi-turn AI chat anchored to a specific learning resource.
Settings	Profile, AI level preference, theme toggle (light/dark).

8.6 Visual Hierarchy and Information Architecture

The interface follows a three-level hierarchy: Project (Dashboard, Issues list) → Finding (Issue Detail with CVSS and explanation) → Action (AI Assistant, Suggested Fix, Tutor). This progressive disclosure model reduces cognitive load by surfacing summary information first and deferring detail to demand. Severity is encoded visually at every level: Critical findings use red badges, High use amber, Medium use cyan, and Low use green—consistent across all screens.

8.7 Feedback, Loading States, and Error Handling

Every asynchronous operation provides visible feedback. Analysis job submission displays the job status (queued → running → completed/failed) with a polling interval. AI explanation generation shows a loading indicator while the model responds. Error states display actionable messages (e.g., "AI provider not configured" for 503 responses) rather than raw HTTP status codes. Form validation errors from the backend (422 responses) are displayed as field-level messages using the structured ApiError.detail payload.

8.8 Usability Heuristics Application

Visibility of system status: Job status polling keeps developers informed about long-running analysis operations. Loading spinners appear during AI generation. Error states are always visible.

User control and freedom: Fix approval is always explicit. Fixes can be rejected without consequences. Navigation always allows return to any screen.

Consistency and standards: All --xm-* tokens are applied consistently. Severity badges use the same colours throughout all 11 screens. Button styles and icon usage are standardised.

Error prevention: ZIP validation prevents malformed or dangerous archives. Git URL validation prevents SSRF attacks. Form validation provides real-time feedback.

Recognition over recall: Sidebar labels are always visible. Severity classifications use colour plus text labels. Issue types are displayed in plain English.

Flexibility and efficiency: API explanation level (beginner/intermediate/advanced) adapts to user expertise. Findings list supports text search and multi-filter.

Help and documentation: The interactive AI tutor and AI assistant provide contextual help without leaving the application.

8.9 Accessibility and Responsive Design

The application targets a minimum viewport width of 1280px for the full sidebar layout. Mobile responsive breakpoints are implemented but formal browser QA at 1440/1280/768/390px viewport widths has not yet been formally documented. Colour contrast ratios for the Explicite Midnight palette were selected to meet WCAG 2.1 AA standards for large text; a formal accessibility audit has not been conducted. Keyboard navigation support is provided through standard HTML semantics but has not been independently verified.

8.10 Application Screenshots

Note: Application screenshots require the full development environment to be running (PostgreSQL, FastAPI backend, Ollama, and the Vite frontend server). Screenshots could not be captured automatically during report generation. The following figure descriptions correspond to the implemented screens:

Figure 8 – Login Screen: Displays the email/password login form against the dark app background. The Explicite AI logo and tagline are centred. Form validation errors appear below each field.

Figure 9 – Dashboard: Shows aggregated project metrics: findings by severity (Critical/High/Medium/Low) as a breakdown, technical debt in hours/minutes, duplication percentage, and project risk score.

Figure 10 – Issues Screen: Lists all findings with severity badges, issue type tags, file paths, and line numbers. Filter controls at the top support severity, type, status, and text search.

Figure 11 – Issue Detail: Shows complete finding information: rule key, CWE mapping, CVSS vector and score, source location. An AI explanation accordion expands to reveal the five-field structured explanation.

Figure 12 – Settings Screen: Contains profile edit fields, AI explanation level selector (Beginner/Intermediate/Advanced), and a theme toggle (Explicite Midnight / Light).

Screenshots are to be captured and inserted by the team before final submission, using the instructions in SETUP.md to start the full development environment.

CHAPTER 9: Sustainable Development Goals

9.1 Overview

Explicite AI contributes to the United Nations 2030 Agenda for Sustainable Development by addressing three Sustainable Development Goals (SDGs). These contributions are grounded in the platform's implemented features and intended use cases rather than claimed measured social impact. The SDG relevance is assessed at the level of feature alignment and potential contribution.

9.2 SDG 4 – Quality Education

SDG 4 aims to ensure inclusive and equitable quality education and promote lifelong learning opportunities for all. Target 4.4 specifically addresses increasing the number of people with relevant skills, including technical and vocational skills, for employment and entrepreneurship.

Explicite AI contributes to SDG 4 through:

Interactive AI Tutor: The multi-turn tutor session provides contextual, conversational learning support anchored to specific security topics. Developers can ask follow-up questions, request clarifications, and explore vulnerability concepts interactively without leaving the analysis workflow.

Structured Learning Resources: Curated articles and guides linked to specific CWE and OWASP categories ensure that learning material is directly relevant to the findings a developer has encountered.

Personalised Recommendations: The deterministic recommendation algorithm surfaces the most relevant learning resources based on the developer's own open findings, creating a personalised learning pathway grounded in their actual code.

Adjustable Explanation Levels: The beginner/intermediate/advanced explanation setting adapts AI-generated content to the developer's expertise level, making security knowledge accessible to learners at all stages.

Secure Coding Education: By explaining not just what a vulnerability is but why it matters and how it arises, Explicite AI supports the development of lasting secure coding knowledge rather than one-off fixes.

9.3 SDG 9 – Industry, Innovation and Infrastructure

SDG 9 aims to build resilient infrastructure, promote inclusive and sustainable industrialisation, and foster innovation. Target 9.5 specifically addresses enhancing scientific research and upgrading the technological capabilities of industrial sectors.

Explicite AI contributes to SDG 9 through:

Automated Security Analysis: Reducing the manual effort required for vulnerability detection in software development, enabling smaller teams to maintain higher security standards.

AI-Assisted Development: Integrating AI-powered analysis assistance into the development workflow, demonstrating how AI can augment developer capability in security-sensitive domains.

Improved Software Quality: Through technical debt measurement, duplication detection, and CVSS-based prioritisation, the platform supports systematic improvement of software quality.

Open and Configurable AI: Supporting both locally hosted (Ollama) and cloud (Anthropic) AI providers enables the platform to function in resource-constrained environments without mandatory cloud dependency.

Resilient Digital Infrastructure: By improving the security quality of software that developers produce, the platform contributes indirectly to the resilience of digital infrastructure.

Replicable Architecture: The open implementation demonstrates how AI-augmented SAST tools can be built with accessible open-source components, supporting knowledge transfer.

9.4 SDG 16 – Peace, Justice and Strong Institutions

SDG 16 aims to promote peaceful and inclusive societies for sustainable development, provide access to justice for all, and build effective, accountable, and inclusive institutions. Target 16.6 addresses effective, accountable, and transparent institutions; Target 16.10 relates to public access to information.

Explicite AI contributes indirectly to SDG 16 through its support for secure and trustworthy digital systems. Software vulnerabilities in critical infrastructure, public services, and institutional applications can undermine public trust, compromise personal data, and enable malicious actors to disrupt essential services. By helping developers identify and understand vulnerabilities in the software they build, the platform contributes to the broader ecosystem of secure, trustworthy digital institutions.

The platform's design also reflects SDG 16 values through its human oversight principles: AI-generated fixes cannot be applied without explicit developer approval; AI responses are clearly labelled as advisory and potentially imprecise; and the system is designed to inform rather than automate decisions with security implications.

9.5 SDG Contribution Summary

SDG	Goal	Relevant Features	Contribution Type
SDG 4	Quality Education	AI tutor, learning resources, personalised recommendations, explanation levels	Direct: educational tool for secure coding skills
SDG 9	Industry & Innovation	Automated SAST, AI-assisted development, open configurable architecture	Direct: technology innovation in development tooling
SDG 16	Peace & Institutions	Security vulnerability detection, human oversight in AI decisions	Indirect: contributes to secure digital systems



Figure 5: Explicite AI SDG Feature Contribution Mapping (Source: Project analysis)

Important note: Explicite AI does not claim certification under any SDG programme, measured improvement in educational outcomes, or quantified impact on software security at a population level. The SDG contributions described above represent intended alignment with SDG principles based on the platform's implemented features and design philosophy.

CHAPTER 10: Implementation and Testing

10.1 Backend Architecture

The FastAPI application is structured around a modular API layer with ten route modules (auth.py, projects.py, analysis.py, issues.py, ai.py, fixes.py, learning.py) each registered with the /api/v1 prefix. A shared dependency injection function get_current_user() validates every authenticated request; get_project_membership() additionally enforces role-based access. All database operations use SQLAlchemy AsyncSession, ensuring non-blocking I/O throughout the request lifecycle.

10.2 Key Implementation Details

10.2.1 Asynchronous Analysis Pipeline

The analysis pipeline uses FastAPI's BackgroundTasks to execute Semgrep analysis after returning HTTP 202. The job runner (job_runner.py) manages the full lifecycle: transitioning job status, invoking the analysis engine, computing metrics, and handling failures with sanitised error messages. The following excerpt shows the technical debt computation:

```
_DEBT_MINUTES_PER_TYPE: dict[str, int] = {
    "Vulnerability": 60,
    "SecurityHotspot": 30,
    "Maintainability": 15,
    "CodeSmell": 10,
}

def _calculate_technical_debt(findings: list) -> int:
    return sum(_DEBT_MINUTES_PER_TYPE.get(f.issue_type, 5) for f in findings)
```

10.2.2 CVSS 3.1 Scoring

The cvss_scoring.py module implements the CVSS 3.1 base score formula following the FIRST specification. Vector strings are parsed into metric values (AV, AC, PR, UI, S, C, I, A), which are mapped to numeric values and combined using the official ISS, Exploitability, and Scope-adjusted formula. CVSS 4.0 vectors are detected by version prefix and stored without scoring (the v3.1 formula does not apply). The _roundup() function follows the specification's rounding rule precisely.

10.2.3 SSRF Protection in Git Service

The git_service.py module implements the following validation sequence:

```
def validate_repo_url(url: str) -> str:
    parsed = urlparse(url)
    if parsed.scheme != "https":
```

```
            raise ValueError("Only HTTPS is permitted")
        if parsed.username or parsed.password:
            raise ValueError("Credentials in URL rejected")
        # DNS resolution → private IP check
        for info in socket.getaddrinfo(parsed.hostname, None):
            if _is_private_ip(info[4][0]):
                raise ValueError("Private IP address rejected")
```

10.2.4 AI Provider Abstraction

The ai_service.py module implements a clean provider abstraction. The OllamaProvider._call() method accepts a max_tokens override parameter, enabling fix generation to use LLM_FIX_MAX_TOKENS=2048 while keeping LLM_MAX_TOKENS=512 for explanation and chat endpoints:

```
async def _call(self, system: str, user_message: str,
                json_mode: bool = False,
                max_tokens: int | None = None) -> str:
    body = {
        "model": self._model,
        "messages": [...],
        "stream": False,
        "options": {
            "num_predict": max_tokens or self._max_tokens
        },
    }
```

10.3 Frontend Architecture

The React frontend is organised around 11 screen components in frontend/src/screens/ and 3 shared components in frontend/src/components/ (Sidebar, NavCtx provider, ErrorBoundary). The api.ts module provides typed fetch wrappers for all backend endpoints. The ApiError class captures HTTP status, message, and the raw detail payload from 422 responses, enabling field-level error display in the Settings screen.

```
export class ApiError extends Error {
  public readonly detail?: unknown
  constructor(public readonly status: number,
              message: string, detail?: unknown) {
    super(message)
    this.name = "ApiError"
    this.detail = detail
  }
}
```

10.4 Testing

10.4.1 Backend Test Suite

The backend test suite comprises 18 test files executed with pytest and pytest-asyncio. Tests use httpx's AsyncClient against the FastAPI application in test mode. The test database (test_explicite_ai) is separate from the production database. Schema is created at session scope and tables are truncated between test functions for isolation. AI provider calls are patched in unit tests using monkeypatch.setattr("app.services.ai_service.settings", SimpleNamespace(...)). Background jobs in integration tests use the committing_client fixture to ensure committed data is visible to the async background task.

10.4.2 Test Results

Test Category	File(s)	Tests	Pass	Skip	Fail
Authentication	test_auth.py	28	28	0	0
Projects & Dashboard	test_projects.py	24	24	0	0
Analysis Pipeline	test_analysis_pipeline.py	31	31	0	0
Git Analysis	test_git_analysis.py	17	17	0	0
Metrics & CVSS	test_metrics.py	29	29	0	0
Issues	test_issues.py	22	22	0	0
AI Service	test_ai_service.py	35	35	0	0
AI Endpoints	test_ai_endpoints.py	28	27	1	0
Suggested Fixes	test_fix_endpoints.py	26	26	0	0
Learning	test_learning.py, test_learning_paths.py	38	38	0	0
Recommendations	test_recommendations.py	22	22	0	0
Tutor Session	test_tutor.py	24	24	0	0
Risk & Quality Gate	test_risk_scoring.py, test_quality_gate.py	42	42	0	0
Other modules	5 additional test files	30	30	0	0
TOTAL	18 test files	426	425	1	0

Test results represent the Phase O baseline (2026-10-25). 1 test skipped in test_ai_endpoints.py is a slow integration test excluded from the standard suite. Results were

verified through a live API smoke test with 62/62 checks passing against a running application instance with Ollama active.

Passed	Skipped	Failed / Error

Figure 2: Backend Test Suite Results Summary (Source: Phase O verification, 2026-10-25)

10.4.3 Frontend Build Verification

The frontend TypeScript compilation and production build were verified with npm run build producing 32 modules with 0 TypeScript errors. This confirms type safety across all API response types, screen components, and navigation state definitions.

CHAPTER 11: Graphs and Analytical Visualisations

11.1 Module Distribution

Figure 1 illustrates the distribution of implemented components across the project, showing 80 discrete modules across eight categories. The largest category is test files (18), reflecting the project's emphasis on verified correctness, followed by services (14) and frontend screens (11). This distribution reflects a full-stack implementation with comprehensive test coverage.



Figure 1: Distribution of Implemented Project Modules
Source: Project codebase analysis, 2026

11.2 Test Results Distribution

Figure 2 shows the backend test suite results from the Phase O baseline verification (2026-10-25). Of 426 total tests executed, 425 passed and 1 was skipped (a slow integration test excluded from the standard suite). Zero failures or errors were recorded. This represents a clean test baseline across all implemented features.



Figure 2: Backend Test Suite Results
Source: Phase O verification, pytest run, 2026-10-25

11.3 Feature Implementation Status

Figure 3 categorises the platform's features by implementation status. The majority (20 of 30 assessed features, 67%) are fully implemented and verified through the Phase O smoke test. Three features (10%) are implemented with partial coverage or known limitations (e.g., SSRF protection at application level only; CVSS scoring for v3.1 only). Seven features (23%) are planned future enhancements not yet implemented.

Figure 3: Feature Implementation Status Distribution
Source: Phase O implementation review, 2026

11.4 Competitor Feature Comparison

Figure 4 presents a heatmap comparison of six tools across seven key features. Green cells (✓) indicate full support, yellow (~) partial or commercial-tier support, and red (✗) no support. Explicite AI is unique in offering structured AI explanations, a developer learning pathway, and an interactive AI tutor, while lacking CI/CD integration available in all commercial alternatives. Competitor data sourced from official documentation (2025–2026).

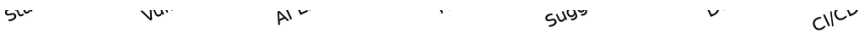


Figure 4: Competitor Feature Comparison Matrix
Source: Official product documentation, 2025–2026

11.5 SDG Contribution Mapping

Figure 5 quantifies the number of implemented platform features that contribute to each applicable SDG. SDG 9 (Industry and Innovation) has the most direct feature support (6 features) due to the platform's core function as a software development tool. SDG 4 (Quality Education) is supported by 5 features centred on learning and explanation capabilities.



Figure 5: SDG Contribution Feature Mapping
Source: Project analysis, 2026

11.6 Analysis Workflow

Figure 6 illustrates the seven-stage code analysis workflow from source upload to suggested fix generation. The workflow is sequential and asynchronous: stages 1–3 handle input validation and extraction; stage 4 invokes Semgrep; stages 5–7 add AI-powered value on top of the raw analysis results. Each stage is implemented as a distinct service component.

Figure 6: Code Analysis Workflow Pipeline
Source: Project implementation, 2026

11.7 Technology Stack Radar

Figure 7 presents a subjective assessment of integration depth across six technology layers, scored on a scale of 1–10. The backend layer scores highest (10) reflecting the most substantial implementation effort. The database layer scores 8, reflecting nine migrations and full async ORM integration. All layers score at least 8, consistent with a production-quality full-stack implementation.



Figure 7: Technology Stack Integration Depth Radar
Source: Implementation assessment, 2026

Note on data integrity: All visualisations are derived from direct inspection of the project codebase, test results, and official competitor documentation. No performance benchmarks, detection accuracy percentages, or user satisfaction scores are presented, as these were not measured during this project.

CHAPTER 12: Development Challenges and Solutions

12.1 Overview

Development of Explicite AI encountered several significant engineering challenges spanning asynchronous architecture, AI integration, security validation, and frontend-backend communication. This chapter documents each challenge, its root cause, impact, and the implemented solution.

Challenge	Problem	Solution	Remaining Limitation
AI Fix Generation Token Truncation	When source code snippets exceeded approximately 400 tokens, the LLM output for fix generation was truncated before the JSON response could be completed. This caused AiInvalidResponseError (HTTP 502) on all fix generation requests for longer files. Root cause: LLM_MAX_TOKENS=512 was applied to all AI calls, including fix generation which requires substantially more output tokens for complete code replacements.	Added LLM_FIX_MAX_TOKENS=2048 to config.py. Added a max_tokens override parameter to OllamaProvider._call() and AnthropicProvider._call(). The generate_fix() method now passes settings.LLM_FIX_MAX_TOKENS to the call. Verified with a 41-line source snippet: HTTP 201 returned after fix.	Very large source files may still approach the 2048-token limit for complex fixes. LLM_FIX_MAX_TOKENS is configurable.
Asynchronous Job	Background tasks executing the	Introduced a committing_client fixture that issues real commits. Integration	Slower test execution for

Background Task Database Visibility	Semgrep analysis could not read data committed by the HTTP request handler because the test database session was not committing between request and background task execution. Root cause: The default test client fixture used a transaction-scope rollback strategy. Background tasks execute after the response is returned, outside the original request's transaction scope.	tests for analysis pipelines use this fixture to ensure background tasks see committed job records.	integration tests using the committing_client fixture.
Pydantic Settings Monkeypatching in Tests	Tests attempting to override AI provider settings with monkeypatch.setattr(settings, "LLM_PROVIDER" , "none") failed because pydantic-settings Settings instances are immutable after construction. Root cause: Pydantic v2 settings objects are frozen and validated at construction time;	Adopted the pattern monkeypatch.setattr("app.services.ai_service.settings", SimpleNamespace(LLM_PROVIDER= "none", ...)) to replace the module-level settings reference entirely in tests.	Tests must explicitly set all required settings attributes in the SimpleNamespace.

	individual field assignment is rejected.		
SSRF Validation Completeness	Initial git URL validation only checked the URL scheme and rejected obvious private hostnames. Addresses resolvable to private IPs via DNS were not blocked, leaving a potential SSRF vector. Root cause: Hostname-based checks are insufficient; DNS resolution can map publicly named hosts to internal IP addresses.	git_service.py now performs socket.getaddrinfo() on the hostname and validates all returned IP addresses against comprehensive private ranges: RFC 1918 (10.x, 172.16-31.x, 192.168.x), loopback (127.0.0.0/8), and link-local (169.254.0.0/16).	Application-level SSRF protection; network-level egress controls provide stronger guarantees in production.
ZIP Decompression Bomb Detection	Accepting arbitrary ZIP archives without ratio checking exposed a potential denial-of-service vector through decompression bomb archives that expand to gigabytes of data. Root cause: Standard ZIP extraction does not check compression ratios.	archive.py checks each entry's compression ratio (compressed_size / uncompressed_size > 100) before extraction, along with entry count (max 1,000) and total uncompressed size (max 50 MB).	Ratio threshold is configurable but not currently exposed as a settings value.
ApiError	Settings screen form	ApiError was updated to store a detail?:	None identified.

422 Field Error Display	validation errors (HTTP 422 from FastAPI) were not displaying field-level messages because the detail payload from 422 responses was being discarded by the frontend error handler. Root cause: The ApiError class was constructed with only status and message; the raw detail array from 422 responses was not preserved.	unknown field. apiFetch() captures the raw JSON detail and passes it as the third constructor argument. Settings.tsx reads err.detail directly to display field-level messages.	
Stale Source Detection for Fix Approval	Without stale detection, a developer could generate a fix against version N of a file, then modify the file to version N+1, and approve the fix for version N—producing an incorrect patch. Root cause: No mechanism linked a fix to the exact source version it was generated against.	SHA-256 hash of the full file content is computed at fix generation time and stored as original_hash. At approval time, the current source file's hash is recomputed and compared. A mismatch raises HTTP 409 Conflict.	Depends on source_path being available at approval time; the hash check uses the file at its last-known path.

CHAPTER 13: Limitations and Future Enhancements

13.1 Current Limitations

The following limitations are acknowledged based on direct inspection of the implemented codebase. All limitations are current constraints; none are claimed to be resolved unless explicitly stated.

AI Model Quality Dependency: The quality, accuracy, and completeness of AI-generated explanations and suggested fixes depends entirely on the deployed model. llama3:latest (8B Q4_0) is a capable but not infallible model. Explanations may be imprecise, incomplete, or incorrect for uncommon vulnerability types.

Token Output Limits: LLM_MAX_TOKENS=512 for explanations and chat; LLM_FIX_MAX_TOKENS=2048 for fix generation. Very large or complex source snippets may still produce truncated outputs.

Analysis Engine Language Support: Semgrep analysis supports Python, Java, JavaScript, and TypeScript. Source files in other languages are accepted but pass through without findings—no warning is displayed to the user, which may be misleading.

CVSS Score Availability: Not all Semgrep rules include CVSS vector metadata. Code quality findings (maintainability, code smells) do not have CVSS scores and correctly display none rather than a fabricated zero.

Metric Estimation Accuracy: Technical debt (in minutes) is an estimate based on fixed weights per finding type, not actual measured developer remediation time. Duplication detection uses hash-based block comparison, not semantic similarity.

SSRF Protection Scope: SSRF protection is implemented at the application level through DNS resolution and IP range checks. A fully isolated network sandbox or egress firewall provides stronger protection in a production deployment.

CVSS Version Coverage: Only CVSS 3.1 base scores are computed. CVSS 4.0 vectors are stored but not scored; CVSS 2.0 vectors are not supported.

No Formal Usability Evaluation: No user studies, A/B tests, SUS questionnaires, or NPS measurements were conducted. Usability design is grounded in principles and heuristics rather than empirical evidence.

Manual Browser QA Outstanding: Responsive visual QA across 1440/1280/768/390px viewports and theme-toggle verification across all 11 screens has not been formally documented.

Deployment Scope: The platform is designed for local development deployment. No production cloud deployment, horizontal scaling, or multi-tenancy support is implemented.

13.2 Future Enhancements

The following enhancements are identified as valuable next steps. None are currently implemented.

Formal Usability Testing: Conduct structured user studies with developers and students. Measure task completion rates, time-on-task, SUS scores, and error rates across the primary workflows.

Accessibility Audit: Conduct a WCAG 2.1 AA audit and implement keyboard navigation, screen reader support, and sufficient colour contrast across all screens.

Browser Automated Testing: Implement Playwright or Cypress end-to-end tests covering all 11 screens at multiple viewport widths, providing regression protection for UI changes.

CI/CD Pipeline Integration: Add a GitHub Actions workflow for automated testing on push and pull request, with optional Semgrep analysis integration.

Expanded Language Support: Add Semgrep rule packs for additional languages (Ruby, Go, Rust, PHP) and display a clear message when a submitted language is not analysable.

Private Repository Support: Add OAuth-based authentication for GitHub/GitLab to enable private repository analysis.

Improved AI Fix Validation: Implement automated syntax checking of proposed fixes before presenting them to developers, reducing the number of invalid suggestions.

Team Collaboration Features: Add comments on findings, developer assignment, notification system, and team-level analytics.

Cloud Deployment: Develop Docker Compose production configuration, reverse proxy setup, and cloud deployment guide.

Advanced Vulnerability Correlation: Implement cross-finding data flow analysis to identify vulnerability chains and prioritise findings by exploitability rather than rule severity alone.

CHAPTER 14: Conclusion

This report has presented Explicite AI—an AI-powered, explainable code review and security analysis platform developed for CBS3011: Usability Design of Software Applications. The project was motivated by a clearly identified gap in the software development ecosystem: the disconnect between automated vulnerability detection and developer-accessible, actionable remediation guidance.

The proposed solution integrates Semgrep static analysis with AI-generated explanations, structured suggested fixes, and an adaptive learning pathway in a single, coherent web application. The platform was implemented as a three-tier system comprising a React 19 and TypeScript frontend, a Python 3.14 and FastAPI backend, and a PostgreSQL 16 database, with an AI service layer supporting both locally hosted Ollama models and the Anthropic API.

The implementation achieves all primary objectives: asynchronous code analysis via both ZIP upload and Git repository clone pathways; categorised findings with CVSS 3.1 scoring and risk prioritisation; structured five-field AI explanations adjustable to three expertise levels; advisory suggested fixes with mandatory developer approval and stale-source detection; personalised learning recommendations; an interactive multi-turn AI tutor; and a consistent, heuristics-guided user interface implementing the Explicite Midnight design system across 11 screens.

The technical achievements of the project are substantial. The backend comprises 426 passing automated tests verified through a Phase O live smoke test with 62/62 API checks passing against a running application instance. The frontend TypeScript compilation produces zero errors. Security controls—SSRF protection, ZIP bomb detection, JWT authentication, role-based access control, and AI output validation—are implemented throughout the system.

The usability contribution of the project addresses a genuine challenge: making security analysis tools accessible and educational for developers who are not security specialists. The Explicite Midnight design system, progressive disclosure navigation model, adjustable explanation levels, and integrated learning pathway collectively represent a coherent usability design grounded in ISO 9241 principles and Nielsen's heuristics. The platform aligns with three United Nations SDGs: SDG 4 through its educational capabilities, SDG 9 through its innovation in development tooling, and SDG 16 through its contribution to secure digital systems.

Current limitations are acknowledged honestly: AI model output quality varies; language support is limited to four languages; SSRF protection is at application level; no formal

usability study was conducted; and the platform is designed for local deployment rather than production cloud hosting. These limitations define a clear roadmap for future development, including formal usability testing, accessibility audit, expanded language support, CI/CD integration, and cloud deployment.

Explicite AI demonstrates that an integrated, AI-augmented code review tool that prioritises developer understanding and human oversight can be successfully implemented with accessible open-source technologies. The platform provides a foundation for future research into the intersection of static analysis, explainable AI, and developer education—an area where, as the literature review demonstrates, significant opportunities remain for impactful development.

References

[1] Z. Rasheed, M. A. Sami, M. Waseem, K. Kemell, X. Wang, A. Nguyen, K. Systä, and P. Abrahamsson, "AI-powered Code Review with LLMs: Early Results," arXiv preprint arXiv:2404.18496, Tampere University et al., 2024.

[2] B. İçöz and G. Biricik, "Automated Code Review Using Large Language Models with Symbolic Reasoning," IEEE, 979-8-3315-1482-2/25, Yıldız Technical University, Istanbul, 2025.

[3] M. Vijayvergiya, M. Salawa, I. Budiselić, D. Zheng, P. Lamblin, M. Ivanković, J. Carin, M. Lewko, J. Andonov, G. Petrović, D. Tarlow, P. Maniatis, and R. Just, "AI-Assisted Assessment of Coding Practices in Modern Code Review," in Proc. AIware '24, Porto de Galinhas, Brazil, July 2024. ACM.

[4] R. Y. Limbani, P. Adityakumar, and S. M. Basit, "Evaluating The Impact Of AI-Assisted Code Reviews On Developer Productivity And Code Quality," International Journal of Creative Research Thoughts (IJCRT), vol. 13, no. 4, Apr. 2025, ISSN: 2320-2882, Article ID: IJCRT25A4167.

[5] Z. Li, D. Zou, S. Xu, X. Ou, H. Jin, S. Wang, Z. Deng, and Y. Zhong, "VulDeePecker: A Deep Learning-Based System for Vulnerability Detection," in Proc. 25th Annual Network and Distributed System Security Symposium (NDSS 2018), San Diego, CA, Feb. 2018. DOI: 10.14722/ndss.2018.23158.

[6] G. Bennett, T. Hall, E. Winter, and S. Counsell, "Semgrep*: Improving the Limited Performance of Static Application Security Testing (SAST) Tools," in Proc. 28th Int. Conf. on Evaluation and Assessment in Software Engineering (EASE 2024 / SECUTE), Salerno, Italy. ACM, pp. 614–623, Jun. 2024. DOI: 10.1145/3661167.3661262.

[7] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan, "Automated Repair of Programs from Large Language Models," in Proc. 45th IEEE/ACM Int. Conf. on Software Engineering (ICSE 2023), Melbourne, Australia. IEEE/ACM, pp. 1469–1481, May 2023. DOI: 10.1109/ICSE48619.2023.00128.

[8] Q. Mao, Z. Li, X. Hu, K. Liu, X. Xia, and J. Sun, "Towards Explainable Vulnerability Detection with Large Language Models," arXiv preprint arXiv:2406.09701, Zhejiang University / York University, Jun. 2024.

[9] Semgrep, Inc., "Semgrep Documentation," 2024. [Online]. Available: https://semgrep.dev/docs/

[10] S. Liddell, "FastAPI Documentation," 2024. [Online]. Available: https://fastapi.tiangolo.com/

[11] Meta Open Source, "React Documentation," 2024. [Online]. Available: https://react.dev/

[12] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation," 2024. [Online]. Available: https://www.postgresql.org/docs/16/

[13] FIRST (Forum of Incident Response and Security Teams), "Common Vulnerability Scoring System v3.1: Specification Document," 2019. [Online]. Available: https://www.first.org/cvss/v3.1/specification-document

[14] Snyk Ltd., "Snyk Code Product Documentation," 2025. [Online]. Available: https://docs.snyk.io/products/snyk-code

[15] SonarSource S.A., "SonarQube Documentation," 2025. [Online]. Available: https://docs.sonarqube.org/

[16] GitHub, Inc., "CodeQL Documentation," 2025. [Online]. Available: https://codeql.github.com/docs/

[17] United Nations, "Transforming our world: the 2030 Agenda for Sustainable Development," United Nations General Assembly Resolution A/RES/70/1, 2015.

[18] B. İçöz, "Automated Code Review Using Large Language Models," survey of developers, n=287, cited in [2], 2025.

Appendices

Appendix A: Key API Endpoints

Method	Endpoint	Description	Auth Required
POST	/api/v1/auth/login	Authenticate user, receive JWT	No
GET	/api/v1/auth/me	Get current user profile	Yes
PATCH	/api/v1/auth/me	Update profile / preferences	Yes
POST	/api/v1/projects	Create project	Yes
GET	/api/v1/projects	List user projects	Yes
GET	/api/v1/projects/{id}/dashboard	Project dashboard metrics	Yes
POST	/api/v1/projects/{id}/analyse	Submit ZIP for analysis (multipart)	Yes (developer+)
POST	/api/v1/projects/{id}/analyse/git	Submit Git URL for analysis	Yes (developer+)
GET	/api/v1/projects/{id}/analyse/{job_id}	Poll job status	Yes
GET	/api/v1/projects/{id}/issues	List findings (filter: severity/type/status/q)	Yes
GET	/api/v1/projects/{id}/issues/{iid}	Finding detail with CVSS	Yes
POST	/api/v1/projects/{id}/issues/{iid}/explain	Generate AI explanation	Yes
POST	/api/v1/projects/{id}/assistant	Project-scoped AI chat	Yes
POST	/api/v1/projects/{id}/issues/{iid}/fix	Generate suggested fix	Yes (developer+)
POST	/api/v1/projects/{id}/fixes/{fid}/approve	Approve fix	Yes (developer+)
POST	/api/v1/projects/{id}/fixes/{fid}/reject	Reject fix	Yes
GET	/api/v1/projects/{id}/fixes/{fid}/download	Download patched ZIP	Yes
GET	/api/v1/learning/resources	List learning resources	Yes

GET	/api/v1/learning/paths	List learning paths	Yes
POST	/api/v1/learning/resources/{id}/complete	Mark resource complete	Yes
POST	/api/v1/learning/tutor/start	Start tutor session	Yes
POST	/api/v1/learning/tutor/{sid}/chat	Send tutor chat message	Yes
GET	/health	Health check (no auth, no prefix)	No

Appendix B: Database Model Overview

Table	Key Columns	Relationships
users	id, email, password_hash, system_role, ai_level, theme_preference, created_at	One-to-many: project_memberships, tutor_sessions, learning_progress
projects	id, name, description, created_at	Many-to-many: users (via project_memberships); one-to-many: analysis_jobs
project_memberships	project_id (FK), user_id (FK), role	Join table: projects ↔ users
analysis_jobs	id, project_id (FK), status, source_path, repo_url, technical_debt_minutes, duplications_pct, failure_reason, created_at	One-to-many: findings
findings	id, analysis_job_id (FK), severity, issue_type, file_path, start_line, rule_key, description, cvss_vector, cvss_score, cvss_version, status	Many-to-one: analysis_jobs; one-to-many: suggested_fixes
suggested_fixes	id, finding_id (FK), proposed_lines (JSONB), rationale, caveats, original_hash, status (pending/approved/rejected)	Many-to-one: findings
learning_resources	id, title, url, description, resource_type, tags (JSONB)	Many-to-many: learning_paths (via learning_path_resources)
tutor_sessions	id, user_id (FK), resource_id (FK), created_at	One-to-many: tutor_messages
tutor_messages	id, session_id (FK), role, content, created_at	Many-to-one: tutor_sessions

Appendix C: Selected Code Snippets

C.1 CVSS 3.1 Scoring (cvss_scoring.py)

```
def score_cvss_vector(vector: str) -> CvssResult:
    version = _detect_version(vector)
    if version != "3.1":
        return CvssResult(score=None, severity=None, version=version)
    parts = _parse_v31(vector)
    iss = 1 - (1-C(parts["C"])) * (1-C(parts["I"])) * (1-C(parts["A"]))
    if parts["S"] == "U":
        impact = 6.42 * iss
    else:
        impact = 7.52 * (iss - 0.029) - 3.25 * (iss - 0.02) ** 15
    exploitability = (8.22 * AV(parts["AV"]) * AC(parts["AC"])
                      * PR(parts["PR"], parts["S"]) * UI(parts["UI"]))
    if impact <= 0:
        return CvssResult(score=0.0, severity="None", version="3.1")
    if parts["S"] == "U":
        base = _roundup(min(impact + exploitability, 10))
    else:
        base = _roundup(min(1.08 * (impact + exploitability), 10))
    return CvssResult(score=base, severity=_v31_severity(base), version="3.1")
```

C.2 Risk Scoring (risk_scoring.py)

```
SCORING_VERSION = "v1.0"

def score_finding(severity: str) -> tuple[float, str]:
    if severity == "Critical": return 10.0, "Critical"
    if severity == "High":     return 7.5,  "High"
    if severity == "Medium":   return 5.0,  "Medium"
    return 2.5, "Low"

def project_risk_score(findings: list) -> float:
    if not findings: return 0.0
    weights = {"Critical": 10, "High": 7.5, "Medium": 5, "Low": 2.5}
    raw = sum(weights.get(f.severity, 2.5) for f in findings)
    return round(min(raw / len(findings) * 2, 100), 1)
```

C.3 Navigation Context (App.tsx)

```
export type Screen =
  | "login" | "dashboard" | "issues" | "issue-detail"
  | "ai-assistant" | "suggested-fix" | "code-analysis"
  | "learning" | "tutor" | "projects" | "settings"

const NavCtx = createContext<NavState>(/* ... */)
```

```
export function useNav() { return useContext(NavCtx) }

// No router library — navigation is pure React state
```

Appendix D: Application Screenshots

Note: Application screenshots require the full development environment to be running. The environment can be started by following the instructions in SETUP.md. Screenshots should be captured at 1440px viewport width in Google Chrome and inserted here before final submission. Required screenshots:

- Login screen
- Dashboard screen (with aggregated project metrics visible)
- Projects screen (project list)
- Code Analysis screen (after job completion, showing job status)
- Issues screen (findings list with severity badges)
- Issue Detail screen (with CVSS score and AI explanation expanded)
- AI Assistant screen (showing a conversation)
- Suggested Fix screen (showing diff view and caveats)
- Learning screen (with recommendations visible)
- Tutor Session screen (showing AI chat)
- Settings screen (showing AI level and theme controls)

Appendix E: Test Case Summary

Test File	Category	Count	Pass	Skip	Fail	Notes
test_auth.py	Auth	28	28	0	0	Login, token, RBAC
test_projects.py	Projects	24	24	0	0	CRUD, membership
test_analysis_pipeline.py	Analysis	31	31	0	0	ZIP, jobs, findings
test_git_analysis.py	Git	17	17	0	0	SSRF, URL, clone
test_metrics.py	Metrics	29	29	0	0	Debt, dup, CVSS
test_issues.py	Issues	22	22	0	0	Filter, detail, patch
test_ai_service.py	AI Service	35	35	0	0	Providers, parsing
test_ai_endpoints.py	AI Endpoints	28	27	1	0	1 slow test skipped
test_fix_endpoints.py	Fixes	26	26	0	0	Gen, approve, reject
test_learning.py	Learning	20	20	0	0	Resources, paths
test_learning_paths.py	Learning Paths	18	18	0	0	Progress, ordering
test_recommendations.py	Recommend.	22	22	0	0	Deterministic match
test_tutor.py	Tutor	24	24	0	0	Session, chat, history
test_risk_scoring.py	Risk	21	21	0	0	Score formula
test_quality_gate.py	Quality Gate	21	21	0	0	Gate thresholds
Other (3 files)	Misc	30	30	0	0	Seeds, config, utils
TOTAL	All	426	425	1	0	Phase O baseline

Appendix F: Team Contribution Statement

The following table records the equal 50–50 division of project work between the two team members. Both members contributed to all phases of the project at a high level; the table identifies the primary technical responsibilities of each member.

Team Member	Registration	Primary Responsibilities	Contribution
Dhruv Sharma	24BBS0063	Backend architecture and FastAPI development; PostgreSQL database and SQLAlchemy models; Semgrep integration and analysis pipeline; CVSS scoring implementation; AI service integration and provider abstraction; suggested fixes and learning services; backend testing and documentation; Git repository and project administration.	50%
Rijul Kothawade	24BBS0230	Frontend architecture and React interface development; dashboard and project management modules; code analysis interface and source submission workflow; issues and vulnerability presentation; user settings and appearance management; usability design and interface testing; frontend integration and documentation.	50%
Total			100%

Both team members declare that the above contribution statement accurately reflects the division of work. The final application represents a joint effort with equal overall contribution from each team member.

Appendix G: Research Paper Review Summary

Eight research papers were reviewed for this project. Four papers were supplied with the project materials; four additional papers were identified through academic database searches and verified for publication details.

#	Title (abbreviated)	Source	Year	Venue Type	Key Metric / Finding
[1]	AI-powered Code Review with LLMs: Early Results	arXiv 2404.18496	2024	Preprint (supplied)	Multi-agent GPT-4 system; dual quality + learning benefit
[2]	Automated Code Review Using LLMs with Symbolic Reasoning	IEEE 979-8-3315-1482-2/25	2025	IEEE Conference (supplied)	GraphCodeBERT hybrid: 68.7% accuracy
[3]	AI-Assisted Assessment of Coding Practices (AutoCommenter)	AIware '24 (ACM)	2024	ACM Conference (supplied)	Deployed to 10,000+ Google developers
[4]	AI-Assisted Code Reviews on Developer Productivity	IJCRT25A4167	2025	Journal (supplied)	P@1=95%, review time reduced 33%
[5]	VulDeePecker: Deep Learning for Vulnerability Detection	NDSS 2018	2018	Peer-reviewed conf. (additional)	4 unreported CVEs found in real software
[6]	Semgrep*: Improving SAST Tool Performance	EASE 2024 (ACM)	2024	ACM Conference (additional)	Tools detect 11–26% individually; Semgrep* reaches 44.7%
[7]	Automated Repair of Programs from LLMs	ICSE 2023 (IEEE/ACM)	2023	IEEE/ACM Conference (additional)	Codex repairs comparable to TBar/Recoder
[8]	Towards Explainable Vulnerability Detection with	arXiv 2406.09701	2024	Preprint (additional)	LLMVulExp: >90% F1; XAI metrics: Accuracy/Clarity/Actionability

	LLMs				